SCALABLE TWITTER CLONE ARCHITECTURE AND STORAGE SYSTEM

System Documentation and Technical Implementation Report

---

1. PROJECT SPECIFICATIONS AND ABSTRACT

The **Scalable Twitter Clone** is an enterprise-grade full-stack social networking platform engineered on top of the **Spring MVC** design pattern. The platform is architected to handle distributed user authentication, asynchronous data publication, multi-user timeline aggregation, and dynamic relationship graph mappings.

The backend infrastructure utilizes **jOOQ** for compile-time type-safe SQL query generation, interacting directly with a persistent **PostgreSQL** relational database engine. By implementing a cookie-based session verification layer alongside custom HandlerInterceptor filters, the platform secures private data spaces (such as user welcome streams and recommendation networks) without incurring stateful memory overhead.

---

2. PROJECT DIRECTORY ARCHITECTURE

The codebase adheres to a strict Maven compilation framework directory tree structure, cleanly decoupling application logic, database access objects, model entities, and frontend templates:

- **tech.twitter.controller**: Manages inbound HTTP traffic boundaries. Contains MainController, LoginController, NonAPIController, and helper modules like ControllerUtils to handle request parsing.
- **tech.twitter.database**: Manages connection pooling and reflection mappings. Houses DatabaseConnectionPool, DatabaseUtil, and GenericDB for dynamic object-relational mapping.
- **tech.twitter.entity**: Defines core data transport schemas, including Member, MemberBase, and SignupResponse.
- **src/main/webapp/WEB-INF/**: Hosts view templates (signup.jsp, welcome.jsp) alongside Spring container maps (mvc-dispatcher-servlet.xml, web.xml).

Structure:

ScalableTwitterClone/

├── .idea/

├── .mvn/

├── images/

```
├── src/
│   ├── generated-sources/
│   │   └── jooq/
│   │       └── tech.twitter/
│   │           ├── tables/
│   │           ├── Keys.java
│   │           ├── Public.java
│   │           ├── Sequences.java
│   │           └── Tables.java
│   ├── main/
│   │   ├── java/
│   │   │   └── tech.twitter/
│   │   │       ├── controller/
│   │   │       ├── database/
│   │   │       ├── entity/
│   │   │       └── global/
│   │   ├── resources/
│   │   └── webapp/
│   │       └── WEB-INF/
│   │           ├── pages/
│   │           ├── mvc-dispatcher-servlet.xml
│   │           └── web.xml
│   └── test/
├── pom.xml
└── .gitignore
```

---

3. CORE DATABASE SCHEMA DESIGN

The relational storage layer is engineered inside PostgreSQL to maintain data normalization and enforce relational integrity constraints. The application utilizes three core tables to map the entire social network platform:

3.1 The member Table

Maintains unique user records, authentication credentials, system roles, and session tokens.

- **Columns**: id (bigint, Primary Key), name (varchar), email (varchar, Unique constraint), role (varchar), password (varchar), image (varchar), token (varchar), is_verified (boolean).

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

	name character varying (100)	email character varying (100)	role character varying (20)	password character varying (30)	image character varying (100)	token character varying (3
1	varri yaswanth	test@gmail.com	cm	pp123	[null]	[null]
2	Sara	sara@gmail.com	cm	ppsara123	[null]	[null]
3	babumohan	babu@gmail.com	cm	babu	[null]	[null]
4	Scarlet johnson	scary@gmail.com	cm	lucy	[null]	[null]
5	alice	alice@gmail.com	cm	test	[null]	[null]
6	bob	bob@gmail.com	cm	test	[null]	[null]
7	peter	peter@gmail.com	cm	test	[null]	[null]
8	varri rohith	rohi@gmail.com	cm	123456	[null]	[null]
9	Sita	sita@gmail.com	cm	sita123	[null]	[null]
10	Lakshman	lucky@gmail.com	cm	test	[null]	[null]
11	Ramudu	ram@gmail.com	cm	sita	[null]	[null]
12	Hamza ali mazari	spy@gmail.com	cm	yalina	[null]	[null]

3.2 The tweet Table

Stores chronological message posts bound to specific profile authors via foreign key constraints.

- **Columns**: id (bigint, Primary Key), message (varchar), created_at (time without time zone), author_id (bigint, Foreign Key referencing member.id).

Data Output Messages Notifications

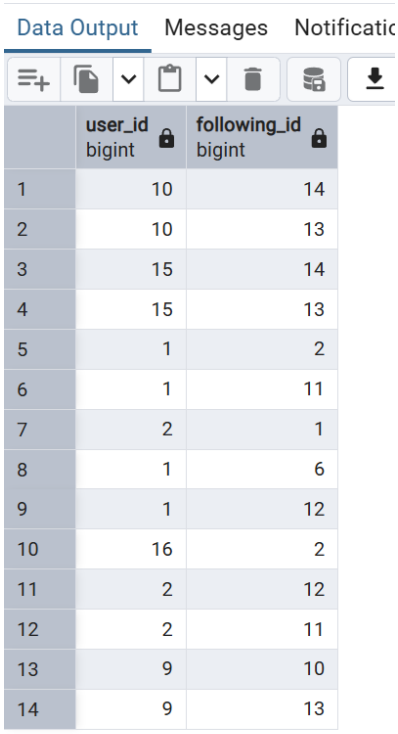
Showing rows: 1 to 10

	message character varying	id bigint	created_at time without time zone	author_id bigint
1	hey this is my first tweet	1	19:29:52.798	6
2	hey macha kinguu	2	23:12:52.5	6
3	Hey my name is babu, Jai BABU.	3	23:16:04.533	6
4	Addicted to the twitter!	4	23:18:26.44	6
5	Tweeting whole day for fun	5	23:20:22.666	6
6	Wohh Alpha movie released in imax	6	23:21:13.64	6
7	Hey i am an english tutuor, free-lancer, Pe...	7	16:04:23.341	11
8	Hey i am sara :)	8	16:49:56.132	2
9	I am Bob	9	19:59:35.907	9
10	It's Uzair Baloch!	10	16:41:14.119	16

3.3 The follower Table

A self-joining bridging table that maps many-to-many relationship states to construct the social graph.

- **Columns**: user_id (bigint, Foreign Key referencing the user), following_id (bigint, Foreign Key referencing the target profile being followed).



Data Output | Messages | Notificatic

	user_id bigint	following_id bigint
1	10	14
2	10	13
3	15	14
4	15	13
5	1	2
6	1	11
7	2	1
8	1	6
9	1	12
10	16	2
11	2	12
12	2	11
13	9	10
14	9	13

---

4. SYSTEM ENVIRONMENT AND BUILD MANAGEMENT

The project handles database compilation pipelines automatically using the jooq-codegen-maven plugin. Profiles are selected to inject structural environment parameters (${DB_URL}, ${DB_USERNAME}, ${DB_PASSWORD}) into the build cycle. This triggers an automated metadata scan of the live PostgreSQL instance to compile runtime table handles into the generated-sources/jooq project path.

---

5. CONTROLLER LOGIC AND ROUTING VIEW ENGINES

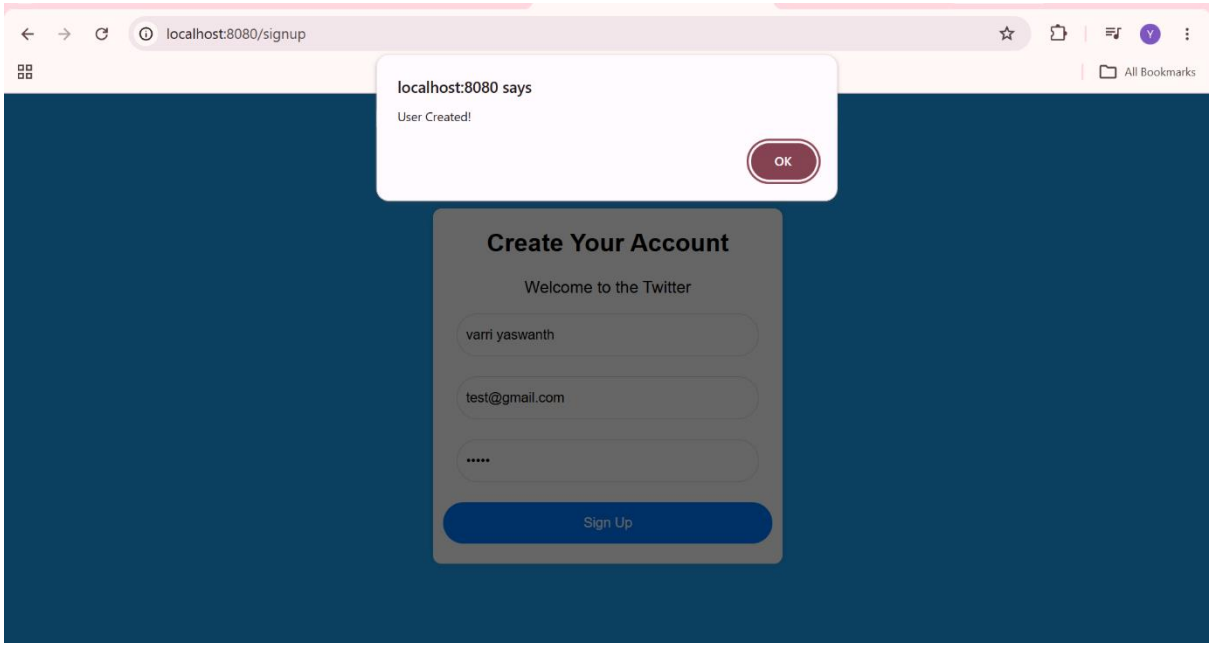
5.1 Asynchronous User Registration

The signup engine uses a POST routing path to validate incoming data structures. When an email passes uniqueness constraints via GenericDB.getCount(), the system hashes credentials, maps the default role (cm), and issues a JSON payload back to the browser:

java

```
if (GenericDB.getCount(tech.twitter.tables.Member.MEMBER,
tech.twitter.tables.Member.MEMBER.EMAIL.eq(member.email)) > 0) {
    message = "User already exists for this email id!";
} else {
    member.role = "cm";
    new GenericDB<Member>().addRow(tech.twitter.tables.Member.MEMBER,
member);
    user_created = true;
    message = "User Created!";
}
```

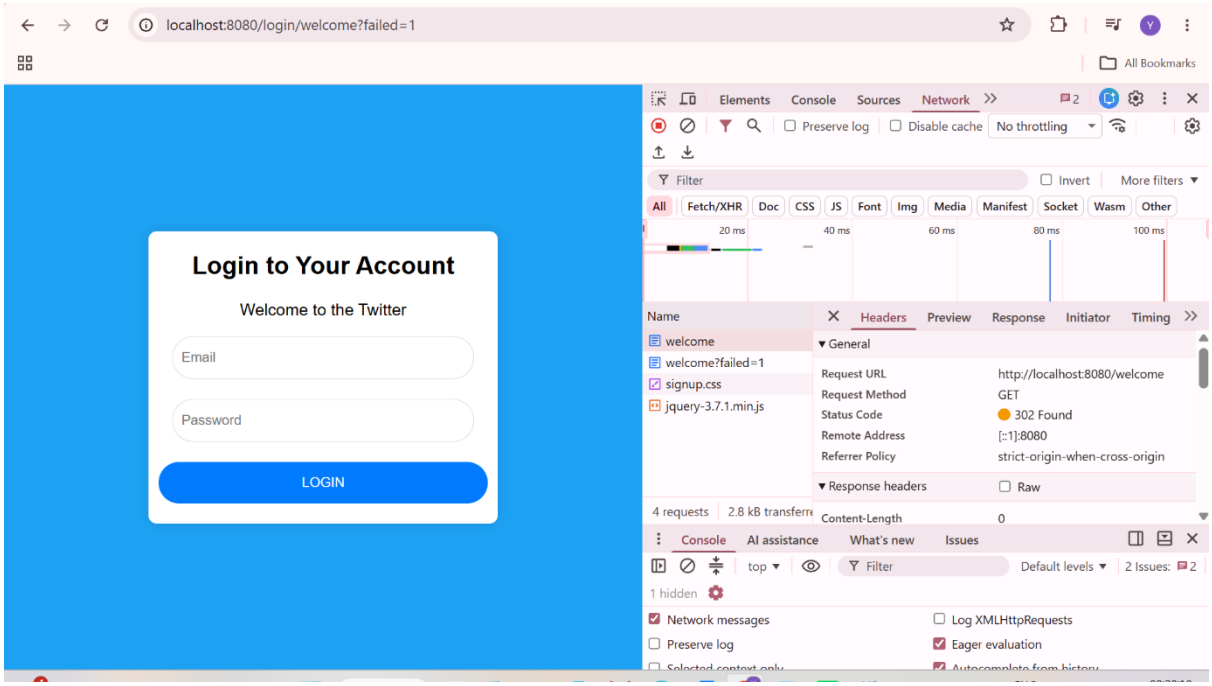
Use code with caution.



5.2 Session Management and State Interception

Secure paths (such as /user/welcome or /user/recommendations) are protected by an interception layer.

- **Active Session**: Evaluates the HttpSession workspace mapping tag "I" to resolve identities instantly.
- **Cookie Fallback**: If the session marker is null, the system scans incoming browser cookies for the token wrapper "t". If found, a background database query re-authenticates the user silently. If both lookups fail, the system issues a 302 Found redirect path to /login/welcome?failed=1.

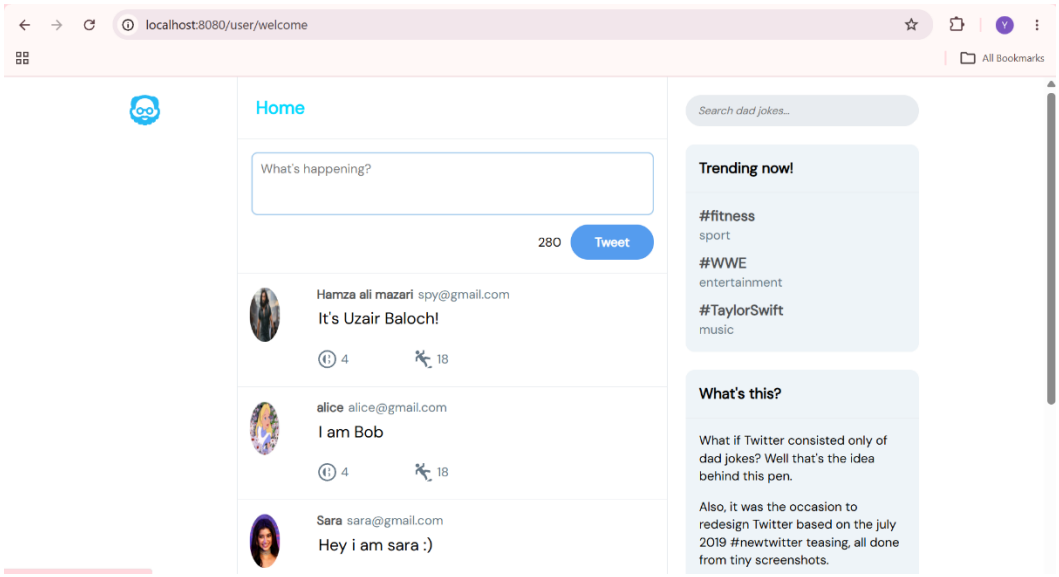


---

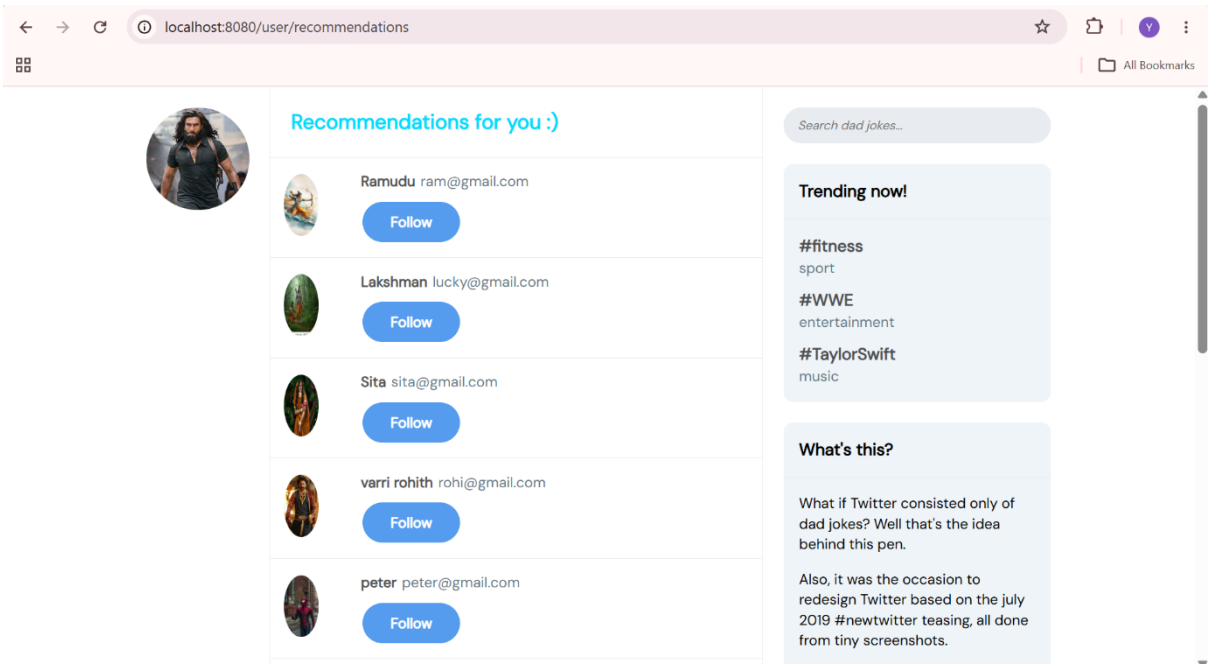
6. FRONTEND PRESENTATION PROOFS AND TIMELINE VIEWS

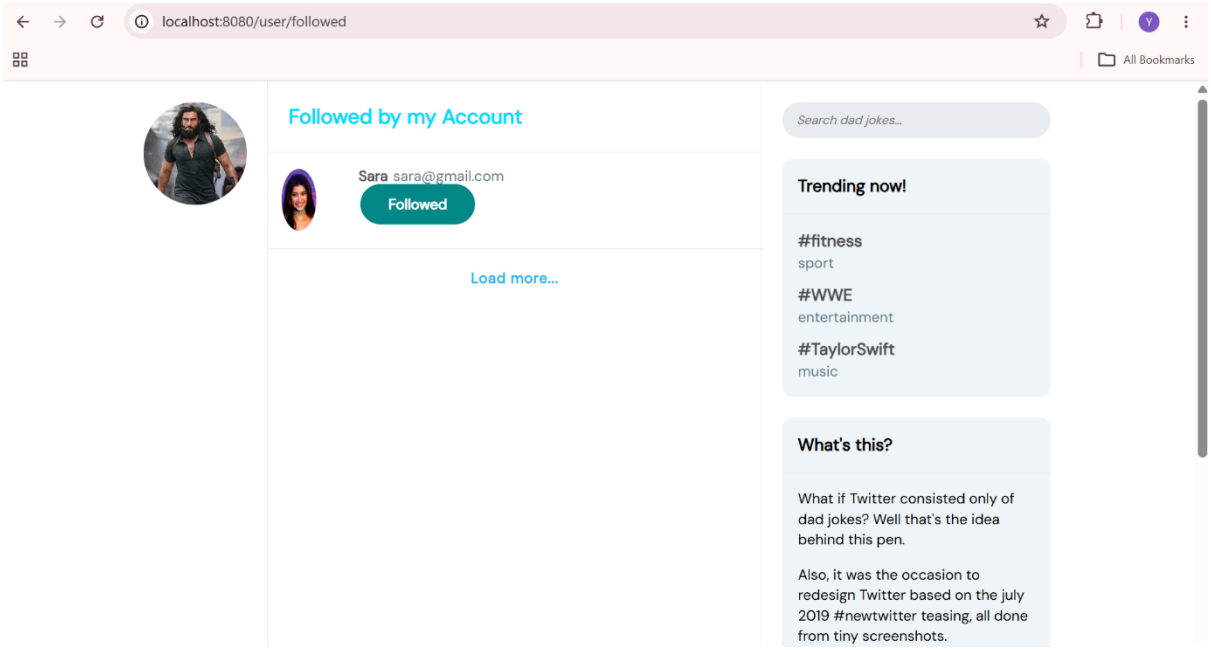
The frontend UI is built using adaptive elements to process and display dynamic backend records:

- **Dynamic Home Timeline Feed**: Assembles the global tweet stream chronologically, rendering post contexts alongside author profiles.



- **Asynchronous Follow Engine**: Employs jQuery AJAX requests to execute relationship toggles. Clicking "Follow" or "Unfollow" modifies data states dynamically and updates button aesthetics across views.





- **Multipart Media File Streaming**: Implements image upload workflows, allowing users to update profile indicators which are then parsed across timelines.